Conceptual Description of the CoreSense Navigation Essential (CS-NE)

Architecture Design Document

1 Introduction to the CoreSense Navigation Essential (CS-NE)

The **CoreSense Navigation Essential (CS-NE)** is a reference architectural design pattern designed to endow autonomous mobile robots with robust, flexible, and self-aware navigation capabilities. Drawing profound inspiration from the EasyNavigation (EasyNav) framework, CS-NE is fundamentally representation-agnostic, lightweight, and highly modular. It leverages EasyNav's philosophy of decoupling navigation logic from the underlying map structures (natively supporting 2D costmaps, 3D Octomaps, NavMaps, and hybrid combinations) and elevates it by embedding it into the CoreSense cognitive architecture.

Within CS-NE, standard navigation components (localizers, planners, and controllers) are encapsulated as *Cognitive Modules* (CogMods). These modules communicate via *Modelets* (semantic knowledge fragments) and are governed by a dual-level Epistemic Control Loop: a base perception-action pipeline for active mission execution, and a metalevel pipeline dedicated to continuous self-awareness, metacognition, and dynamic adaptation.

2 CS-NE Core Components: Modules, Processes, and Modelets

Table **??** defines the primary cognitive modules that constitute the CS-NE base architecture, detailing the processes they execute and the modelets they consume and generate.

Table 1: Main Cognitive Modules, Processes, and Modelets in CS-NE

Cognitive Module (CogMod)	**Primary Process**	**Input Modelets**	**Output Modelets**
Mapping CogMod	Map Generation & Update	LaserScan, PointCloud, RobotPose	OccupancyGrid, ElevationMap, Octomap
Localization CogMod	State Estimation (Probabilistic)	Odometry, LaserScan, BaseMap	RobotPose (with uncertainty metric)
GlobalPlanner CogMod	Path Planning (Search/Optimization)	GoalPose, RobotPose, BaseMap	GlobalPath (Waypoints)
LocalPlanner CogMod	MPC/Trajectory Control	GlobalPath, RobotPose, LocalMap	MotorCmd (Twist/Velocity)
Mission-Aware CogMod	Value & Policy Evaluation	SystemStatus, GoalPose	EvaluatedPath, TaskAlert

3 Architectural Elements: World, Self, and Pipelines

CS-NE strictly separates the operational (base) level from the metacognitive (self-regulatory) level. The entities modeled and processed across these levels are detailed in Table **??**.

4 Use Case Narrative: Building a Mobile Robot with CS-NE

The Scenario: Alice is a robotics engineer tasked with building a new autonomous mobile robot (AMR) for dynamic warehouse logistics. The warehouse features both wide-open static aisles and highly cluttered, dynamic packing zones where pallets are frequently moved.

Step 1: Leveraging Representation-Agnostic Design
Alice begins by instantiating the CS-NE framework. Thanks to its EasyNav-inspired roots, she doesn't have to lock her system into a single map representation. For navigating the wide aisles, she configures the *Mapping CogMod* to use a lightweight 2D NavMap to conserve computational resources. However, for the cluttered packing zones—where forklift forks present vertical hazards—she configures a secondary plugin to seamlessly instantiate and maintain a 3D Octomap.

Step 2: Configuring the Base Pipelines
She wires the base cognitive modules. The LiDAR and wheel encoders feed the *Localization CogMod*, generating a continuous *RobotPose* modelet. When the warehouse fleet manager assigns a task, it enters the *Perception Pipeline* as a *GoalPose*. The *GlobalPlanner CogMod* calculates the optimal route, and the *LocalPlanner CogMod* continuously computes safe, executable *MotorCmds*.

Step 3: The Metacognitive Advantage in Action
During deployment, the AMR enters a highly reflective packing zone, causing LiDAR scans to scatter and drop in quality.

1. **Meta-Perception:** The *Localization CogMod* begins outputting a *RobotPose* modelet with a rapidly degrading certainty metric. The *Meta-Perception Pipeline* detects this anomaly by introspecting the base layer. It updates the *Self Model*, changing the robot's epistemic state to *"Uncertain Localization."*

2. **Meta-Action:** Reacting to the Self Model update, the *Meta-Action Pipeline* intercepts the base execution to dynamically reconfigure the system. It commands the *LocalPlanner CogMod* to reduce its maximum velocity to 0.2 m/s to ensure safety. Simultaneously, it alters the active sensor fusion policy in the *Localization CogMod*, forcing it to rely heavily on wheel odometry and IMU data until the LiDAR confidence recovers.

Conclusion
By utilizing the CoreSense Navigation Essential, Alice rapidly developed a warehouse AMR that transcends standard state-machine logic. She built a highly adaptable, self-aware system capable of modifying its own navigation representations and policies based on runtime context—embodying the true potential of the CoreSense cognitive architecture combined with the pragmatic, modular flexibility of EasyNavigation.

Table 2: Main Elements in World Model, Self Model, and Dual-Level Pipelines

Architectural Domain	Main Elements & Descriptions
World Model	**Static Infrastructure:** Permanent fixtures represented in an agnostic format (e.g., NavMap, GridMap). **Dynamic Entities:** Moving humans or forklifts modeled with velocity vectors and predicted trajectories.
Self Model	**Kinematic Profile:** Robot footprint, max velocity, kinematic constraints. **Epistemic State:** Confidence/variance of current localization, quality of the current map. **Resource State:** Battery level, computation load, active EasyNav plugin configurations.
Perception Pipeline (Base Level)	**Sense → Extract → Model → Integrate → Project.** Translates raw sensor data (e.g., LiDAR) into obstacle features, integrates them into the active World Model, and projects collision probabilities along the planned path.
Action Pipeline (Base Level)	**Trigger → Policy Selection → Enactment → Actuation.** Initiated by a new GoalPose; selects the appropriate planning plugin, generates a path, computes local control efforts, and outputs MotorCmds.
Meta-Perception Pipeline(Metalevel)	Monitors the Base pipelines. Extracts features such as "Localization Variance High" or "Planner Execution Timeout". Integrates these into the Self Model to recognize operational states like "Lost" or "Stuck".
Meta-Action Pipeline (Metalevel)	Triggers architectural reconfiguration. Can hotswap representation plugins (e.g., from 2D costmap to 3D Octomap), adjust maximum velocity constraints, or initiate a recovery behavior (e.g., rotate to re-localize).